dynamic programming, greedy algorithms, divide and conquer, and brute force methods,

1. Dynamic Programming:

- **Concept:**

Dynamic programming solves problems by breaking them down into overlapping subproblems and storing the solutions to these subproblems in a table to avoid redundant calculations. It's a bottom-up approach, starting with the smallest subproblems and combining them to solve larger ones.

- **Key Characteristics:**
    - **Overlapping subproblems:** The problem can be broken down into subproblems that are solved repeatedly.
    - **Optimal substructure:** The optimal solution to the problem can be constructed from optimal solutions to subproblems.
- **Examples:**

Computing binomial coefficients, maximum subarray problem, and many other optimization problems.

- **Notes:**
    - It's highly efficient for optimization problems due to its ability to avoid redundant calculations.
    - Can be implemented using either top-down (memoization) or bottom-up (tabulation) approaches.

2. Greedy Algorithms:

- **Concept:**

Greedy algorithms make locally optimal choices at each step, hoping to find a globally optimal solution. They are typically simple and fast but don't always guarantee the best overall solution.

- **Key Characteristics:**
    - **Making the best immediate choice:** At each step, the algorithm chooses the option that appears to be the best at that moment.
    - **Doesn't look ahead:** The algorithm doesn't consider the potential consequences of its choices beyond the immediate step.
- **Examples:**

Huffman coding, Dijkstra's algorithm, and Activity selection problem.

- **Notes:**
    - Good for problems where a locally optimal choice leads to a globally optimal solution.
    - May not always work for all optimization problems.

3. Divide and Conquer:

- **Concept:**

Divide and conquer algorithms break down a problem into smaller, independent subproblems, solve them recursively, and then combine the solutions to solve the original problem.

- **Key Characteristics:**
    - **Divide:** The problem is divided into smaller subproblems of the same type.
    - **Conquer:** The subproblems are solved recursively.
    - **Combine:** The solutions to the subproblems are combined to solve the original problem.
- **Examples:**

Merge sort, Quick sort, and Strassen's algorithm for matrix multiplication.

- **Notes:**
    - Effective for problems that can be naturally divided into independent subproblems.
    - Can lead to efficient and scalable solutions, especially for large datasets.

4. Brute Force Method:

- **Concept:**

Brute force algorithms try all possible solutions to a problem until a satisfactory solution is found. They are simple but often inefficient, especially for larger problems.

- **Key Characteristics:**
    - **Systematic exploration:** The algorithm systematically explores all possible solutions.
    - **No optimization:** The algorithm doesn't use any shortcuts or heuristics to improve efficiency.

    Example – 8 queen problem

5. Backtracking algorithms

Backtracking algorithms are like problem-solving strategies that help explore different options to find the solution, It is improved brute force technique.

- Work by trying out different paths and if one doesn't work, then backtrack and try another until he right one is found. It's like solving a puzzle by testing different pieces until they fit together perfectly.

- Useful for problems where you must generate all valid combinations, permutations, or subsets under constraints.

Examples: sudoku, 8 queen problem